



PAI-2 BYODSEC - Memoria de Entrega

1. Resumen de decisiones tecnicas y algoritmicas

1.1 Alcance implementado

El proyecto esta implementado como arquitectura cliente-servidor en Python con canal TLS 1.3 obligatorio.

Decisiones principales:

1. Transporte seguro exclusivamente con TLS 1.3.
2. Eliminacion de mecanismos de aplicacion legacy (`nonce` , `HMAC` , modo plano), delegando confidencialidad, autenticidad e integridad en TLS.
3. Credenciales en reposo con `PBKDF2-HMAC-SHA256` + `salt` aleatorio por usuario.
4. Modelo funcional actualizado a mensajes de texto:
 - `MSG` para envio de mensajes.
 - `HISTORY` para consulta de historial.
 - compatibilidad legacy `TX` transformada a `MSG` .
5. Persistencia SQLite con tablas de `users` , `sessions` , `messages` , `login_attempts` y compatibilidad historica.
6. Mitigacion de brute force en login con rate limit + backoff exponencial.

Referencias de implementacion:

- Transporte TLS: `src/common/transport.py` .
- Parametros TLS: `src/server/config.py` (`TLS_MIN_VERSION=1.3` , `TLS_ECDH_CURVE=prime256v1` , `TLS_ALLOWED_CIPHERS`).
- Modelo de mensajes (1..144): `src/common/models.py` .
- Handlers de `MSG` / `HISTORY` : `src/server/handlers.py` .
- Persistencia de mensajes: `src/server/storage.py` .
- API cliente de mensajes: `src/client/api.py` .

1.2 Seguridad aplicada

1. **Confidencialidad del canal:** TLS 1.3 obligatorio.
2. **Autenticidad del servidor:** validacion de certificado en cliente.
3. **Integridad del canal:** garantizada por TLS 1.3.
4. **Credenciales seguras en BD:** hash PBKDF2 con iteraciones endurecidas.
5. **Proteccion en login:** bloqueo por intentos fallidos y backoff.
6. **Sesion:** creacion, expiracion e invalidacion por logout.

1.3 Evidencia de pruebas

Ejecucion local de test realizada el **16/03/2026**:

- Comando: `python -m unittest discover -s tests -p "test_*.py" -v`
- Resultado: **53 tests ejecutados, 53 OK**

Coberturas destacables:

1. Flujo completo registro/login/msg/history/logout (`tests/test_integration.py`).
2. Validacion de longitud 1..144 y errores `EMPTY_MESSAGE` / `MSG_TOO_LONG` (`tests/test_messages.py` , `tests/test_cliente.py`).
3. Seguridad de login (`tests/test_security.py` , `tests/test_rate_limit.py`).
4. Transporte TLS y rechazo de CA invalida (`tests/test_tls_transport_integration.py` , `tests/test_transport_tls.py`).
5. Generacion de certificados ECC/RSA (`tests/test_tls_cert_generation.py`).

Evidencia final de benchmark comparativo (16/03/2026):

- Comando:
`python scripts/run_benchmark_pai1_vs_pai2.py --clients 300 --workers 80 --messages-per-client 2`
- Salidas: `logs/benchmark_pai1_run_1773694240.json` ,
`logs/benchmark_pai2_run_1773694240.json` , `docs/BENCHMARK_COMPARATIVA.md`
- Resultado observado:
 - PAI1 (sin TLS): 231/300 clientes OK, 462 tx, 7.484 msg/s , latencia media 13.439 s , p95 31.763 s .
 - PAI2 (TLS 1.3): 262/300 clientes OK, 524 mensajes, 13.761 msg/s , latencia media 7.786 s , p95 21.852 s .

Nota de alcance para esta entrega:

- No se incluyen trazas de sniffer (`.pcap`) en el entregable actual.

2. Manual de despliegue y uso (Linux)

2.1 Requisitos

1. Linux (Ubuntu/Debian/Fedora o similar).
2. Python 3.10 o superior.
3. `pip`.
4. Opcional para capturas: `tcpdump` y `tshark`.

2.2 Preparacion

Desde raiz del repo:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
cp .env.example .env
```

2.3 Certificados TLS

ECC por defecto:

```
python scripts/generate_tls_certs.py --force
```

RSA opcional:

```
python scripts/generate_tls_certs.py --algorithm rsa --force
```

2.4 Inicializar datos

```
python config/seed_database.py
```

Usuarios semilla (`config/seed_users.json`):

1. `alice / AliceSecure2024!`
2. `bob / BobPassword123#`
3. `admin / AdminPass2024$`

2.5 Ejecucion manual

Terminal 1:

```
source .venv/bin/activate  
python -m src.server.server
```

Terminal 2:

```
source .venv/bin/activate  
python -m src.client.client
```

2.6 Ejecucion con scripts Linux

```
chmod +x scripts/*.sh  
./scripts/start_server.sh  
./scripts/start_client.sh  
./scripts/run_tests.sh  
./scripts/demo_run.sh
```

2.7 Verificacion de canal cifrado (sniffer)

Nota: en esta entrega se documenta la metodologia, pero no se adjuntan trazas `.pcap` ejecutadas.

Captura:

```
sudo tcpdump -i lo -nn -s0 -w logs/tls_capture.pcap tcp port 9999
```

Inspeccion:

```
tshark -r logs/tls_capture.pcap -Y "tls" -T fields -e frame.number \
-e ip.src -e ip.dst -e tls.record.version
```

Criterio esperado:

1. Se observa handshake TLS.
2. No se observan credenciales ni contenido de mensaje en claro.

3. Grado de completitud (trazable)

3.1 Objetivos globales

Objetivo	Estado	Evidencia	Comentario
Canal seguro para credenciales y mensajes con SSL/TLS	Cumplido	<code>src/common/transport.py</code> , <code>src/client/api.py</code> , <code>src/server/server.py</code>	TLS 1.3 forzado cliente/servidor
Cipher suites robustos con TLS 1.3	Cumplido	<code>TLS_ALLOWED_CIPHERS</code> + verificacion de cipher negociado en cliente/servidor	Se aplica politica allowlist sobre el cipher TLS negociado
Analisis de trafico para verificar canal seguro	No entregado en esta version	Procedimiento definido en esta memoria	Se documenta metodologia, pero no se

Objetivo	Estado	Evidencia	Comentario
			anexan .pcap
Soportar ~300 empleados concurrentes	Parcial	Benchmark con 300 clientes en docs/BENCHMARK_COMPARATIVA.md	Se alcanza carga objetivo de prueba, con degradacion y fallos bajo pico
Analisis de rendimiento y escalabilidad con/sin SSL/ TLS	Parcial	scripts/benchmark_tls_capacity.py , scripts/benchmark_pai1_capacity.py , scripts/run_benchmark_pai1_vs_pai2.py , docs/BENCHMARK_COMPARATIVA.md	Comparativa ejecutada y trazable; persisten errores bajo carga
Extra MitM activo (opcional)	Parcial (opcional)	rechazo de CA invalida en tests/test_tls_transport_integration.py	No hay memoria de ataque MitM completo

Resultado sintetico de benchmark (300 clientes)

Metrica	PAI1 (sin TLS)	PAI2 (TLS 1.3)
Clientes OK	231	262
Clientes fallidos	69	38
Mensajes/tx enviados	462	524
Tiempo total (s)	61.734	38.080
Throughput (msg/s)	7.484	13.761
Latencia media por cliente (s)	13.439	7.786

Metrica	PAI1 (sin TLS)	PAI2 (TLS 1.3)
Latencia p95 por cliente (s)	31.763	21.852

Interpretacion:

- En esta corrida concreta, la implementacion TLS obtuvo mejor throughput y menor latencia que el baseline PAI1.
- El objetivo de 300 concurrencias se considero en la prueba, pero ambos escenarios presentan errores bajo pico.

3.2 Requisitos funcionales

Requisito funcional	Estado	Evidencia	Comentario
Registro de usuarios (usuario+contrasena)	Cumplido	<code>handle_register</code> en <code>src/server/handlers.py</code>	
Aviso si usuario ya existe	Cumplido	<code>user_exists</code> + respuesta de error	
No modificar datos tras registro	Cumplido (sin endpoint de edicion)	No hay API de actualizacion de usuario	
Inicio de sesion	Cumplido	<code>handle_login</code> / <code>ClientAPI.login</code>	
Verificar credenciales y denegar invalidas	Cumplido	<code>verify_password</code> en <code>src/common/crypto.py</code>	
Cerrar sesion	Cumplido	<code>handle_logout</code> / <code>ClientAPI.logout</code>	
Usuarios preexistentes	Cumplido	<code>config/seed_users.json</code> , <code>config/seed_database.py</code>	
Envio de mensajes	Cumplido	<code>MSG</code> en	1..144

Requisito funcional	Estado	Evidencia	Comentario
de texto al servidor		<code>src/server/server.py</code> + <code>handle_message</code>	caracteres
Persistencia de usuarios	Cumplido	tabla <code>users</code> en <code>src/server/storage.py</code>	
Persistencia de mensajes + numero por usuario	Cumplido	tabla <code>messages</code> , <code>count_user_messages</code> , <code>HISTORY</code>	Se guarda mensaje y total por usuario
Interfaz por sockets seguros	Cumplido	<code>src/common/protocol.py</code> + TLS	

3.3 Requisitos de informacion

Requisito de informacion	Estado	Evidencia	Comentario
Usuario unico y contraseña	Cumplido	<code>username</code> unico en tabla <code>users</code>	Password en hash+salt
BD inicial con usuarios pre-registrados y sin mensajes previos	Parcial	<code>seed_users.json</code> y <code>seed_database.py</code>	Cumple en BD nueva; si no se limpia BD, puede haber mensajes previos
Historial de mensajes con numero y fecha	Cumplido	<code>get_user_message_history</code> , <code>count_user_messages</code> , campos <code>ts/created_at</code>	
Envio incluye usuario + texto max 144	Cumplido	<code>MESSAGE_MAX_LENGTH=144</code> , validacion cliente/servidor y <code>CHECK SQL</code>	
Mensajes de sistema (registro/login/envio)	Cumplido	respuestas en handlers y API	

3.4 Requisitos de seguridad

Requisito de seguridad	Estado	Evidencia	Comentario
Almacenamiento seguro de credenciales	Cumplido	PBKDF2-HMAC-SHA256 en src/common/crypto.py	
Verificacion segura de credenciales	Cumplido	verify_password + compare_digest	
Proteccion frente a brute force	Cumplido	SecurityManager.check_rate_limit	
Integridad/ confidencialidad/ autenticidad en envio	Cumplido	TLS 1.3 obligatorio	
Integridad de datos en BD	Parcial	restricciones SQL + flujo controlado	No hay firma/ MAC en reposo ni controles de auditoria avanzados